

INFORME DE DESARROLLO Y PRUEBAS

PharmaMobil

Modelado de dominio y pruebas unitarias en Kotlin Multiplatform

Desarrollo de Aplicaciones Móviles · Ciclo VI

Estudiante: Rony

Fecha: 18 de agosto de 2026

Plataformas: Android e iOS

Tecnología: Kotlin Multiplatform y Compose Multiplatform

1. Objetivo de la sesión

Desarrollar el modelo de dominio de PharmaMobil y comprobar mediante pruebas unitarias las reglas principales del negocio: registro del cliente, disponibilidad del producto, cálculo del subtotal, control de stock y total del pedido.

2. Trabajo realizado

- Organización del código compartido en paquetes data, demo y domain.
- Creación de los modelos Cliente, Producto, DetallePedido, Pedido y EstadoPedido.
- Implementación de validaciones para identificadores, DNI, precio, cantidad y stock.
- Creación de casos de uso para subtotal y disponibilidad de inventario.
- Creación de pruebas comunes con kotlin.test para Android e iOS.

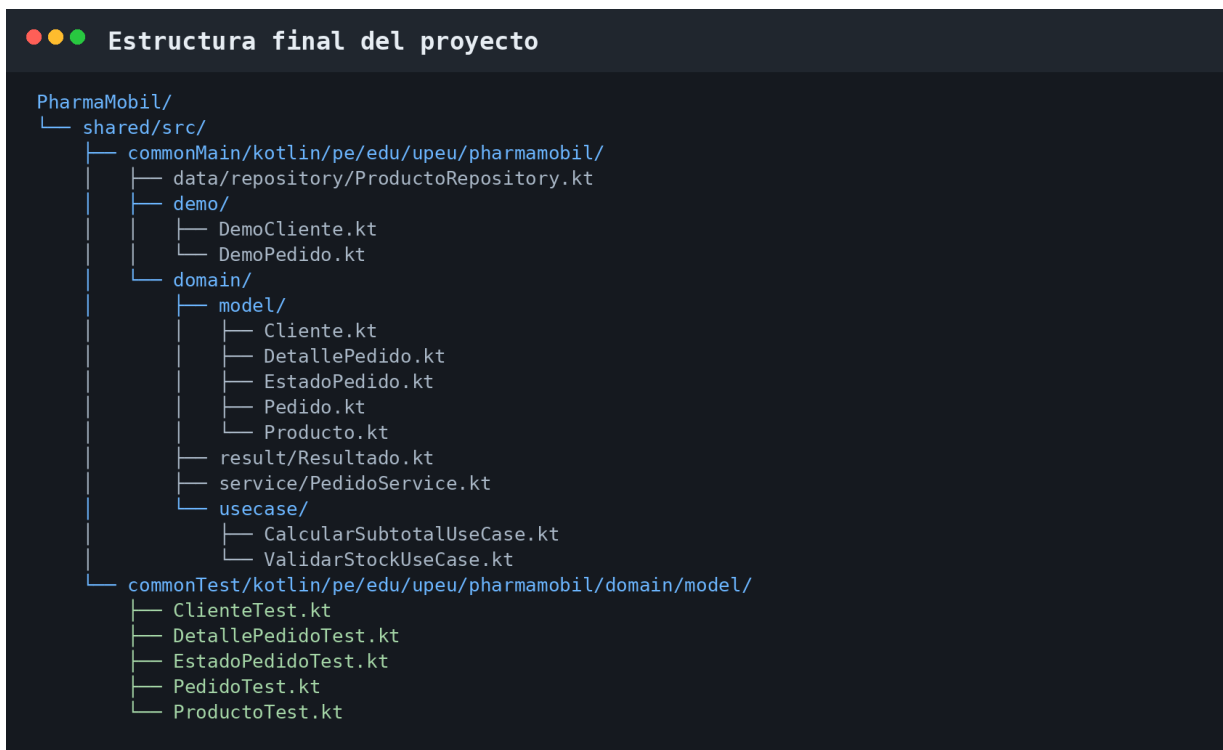


Figura 1. Estructura final del código compartido y sus pruebas.

3. Paquetes y modelo de dominio

La implementación mantiene responsabilidades separadas y evita mezclar la lógica de negocio con la interfaz de usuario.

Componente	Responsabilidad	Regla principal
Cliente	Representa al comprador.	ID positivo, nombre obligatorio y DNI de 8 dígitos.
Producto	Representa un medicamento.	Precio positivo y stock no negativo.
DetallePedido	Relaciona producto y cantidad.	Cantidad positiva y menor o igual al stock.
Pedido	Agrupar cliente y detalles.	Debe contener por lo menos un producto.
EstadoPedido	Controla el ciclo del pedido.	Una cancelación exige un motivo.

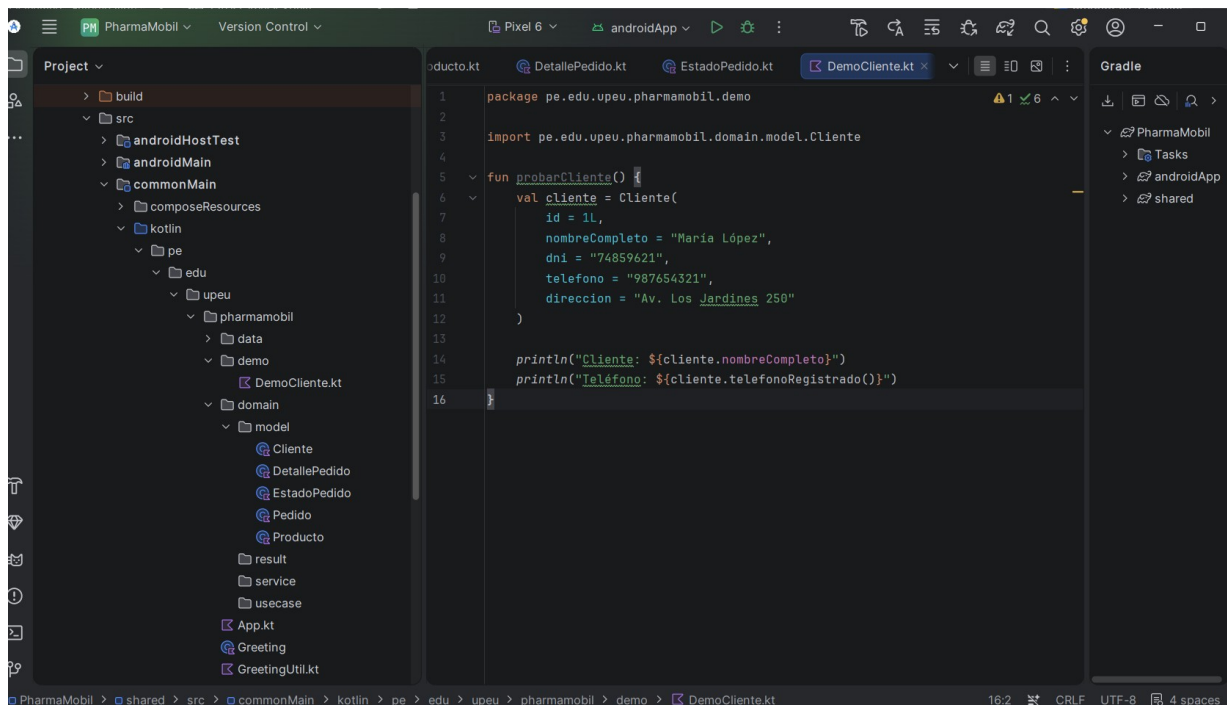


Figura 2. Creación del paquete demo y función probarCliente().

4. Pruebas del subtotal y del stock

La prueba solicitada utiliza un producto de precio S/ 12.50 y una cantidad de 3 unidades. El resultado esperado del subtotal es S/ 37.50. Además, se comprueba que un producto con stock igual a cero no pueda agregarse al pedido.

```

DetallePedidoTest.kt – subtotal y stock

1  package pe.edu.upeu.pharmamobil.domain.model
2
3  import kotlin.test.Test
4  import kotlin.test.assertEquals
5  import kotlin.test.assertFailsWith
6
7  class DetallePedidoTest {
8      @Test
9      fun calculaSubtotalDeUnProducto() {
10         val producto = crearProducto(precio = 12.50, stock = 10)
11         val detalle = DetallePedido(producto = producto, cantidad = 3)
12
13         assertEquals(
14             expected = 37.50,
15             actual = detalle.subtotal(),
16             absoluteTolerance = 0.001
17         )
18     }
19
20     @Test
21     fun rechazaProductoConStockCero() {
22         val producto = crearProducto(precio = 8.90, stock = 0)
23
24         assertFailsWith<IllegalArgumentException> {
25             DetallePedido(producto = producto, cantidad = 1)
26         }
27     }
28
29     @Test
30     fun rechazaCantidadMayorAlStock() {
31         val producto = crearProducto(precio = 5.00, stock = 2)
32
33         assertFailsWith<IllegalArgumentException> {
34             DetallePedido(producto = producto, cantidad = 3)
35         }
36     }
37
38     private fun crearProducto(precio: Double, stock: Int) = Producto(
39         id = 101L,
40         nombre = "Paracetamol 500 mg",
41         laboratorio = "Medifarma",
42         precio = precio,
43         stock = stock,
44         requiereReceta = false
45     )
46 }

```

Figura 3. Pruebas de cálculo del subtotal y rechazo por falta de stock.

5. Pruebas de disponibilidad del producto

La función `hayStock(cantidad)` solo devuelve verdadero cuando la cantidad solicitada es positiva y el inventario disponible es suficiente.

```
●●● ProductoTest.kt – disponibilidad
1 package pe.edu.upeu.pharmamobil.domain.model
2
3 import kotlin.test.Test
4 import kotlin.test.assertFalse
5 import kotlin.test.assertTrue
6
7 class ProductoTest {
8     @Test
9     fun productoConStockCeroNoEstaDisponible() {
10         val producto = crearProducto(stock = 0)
11
12         assertFalse(producto.hayStock(cantidad = 1))
13     }
14
15     @Test
16     fun productoConStockSuficienteEstaDisponible() {
17         val producto = crearProducto(stock = 10)
18
19         assertTrue(producto.hayStock(cantidad = 3))
20     }
21
22     @Test
23     fun cantidadCeroNoEsUnaSolicitudValida() {
24         val producto = crearProducto(stock = 10)
25
26         assertFalse(producto.hayStock(cantidad = 0))
27     }
28
29     private fun crearProducto(stock: Int) = Producto(
30         id = 101L,
31         nombre = "Paracetamol 500 mg",
32         laboratorio = "Medifarma",
33         precio = 12.50,
34         stock = stock,
35         requiereReceta = false
36     )
37 }
```

Figura 4. Pruebas para stock cero, stock suficiente y cantidad inválida.

6. Organización de commonTest

Las pruebas están separadas del código de producción. commonMain contiene la implementación compartida y commonTest contiene los casos que pueden ejecutarse con los corredores de prueba de cada plataforma.

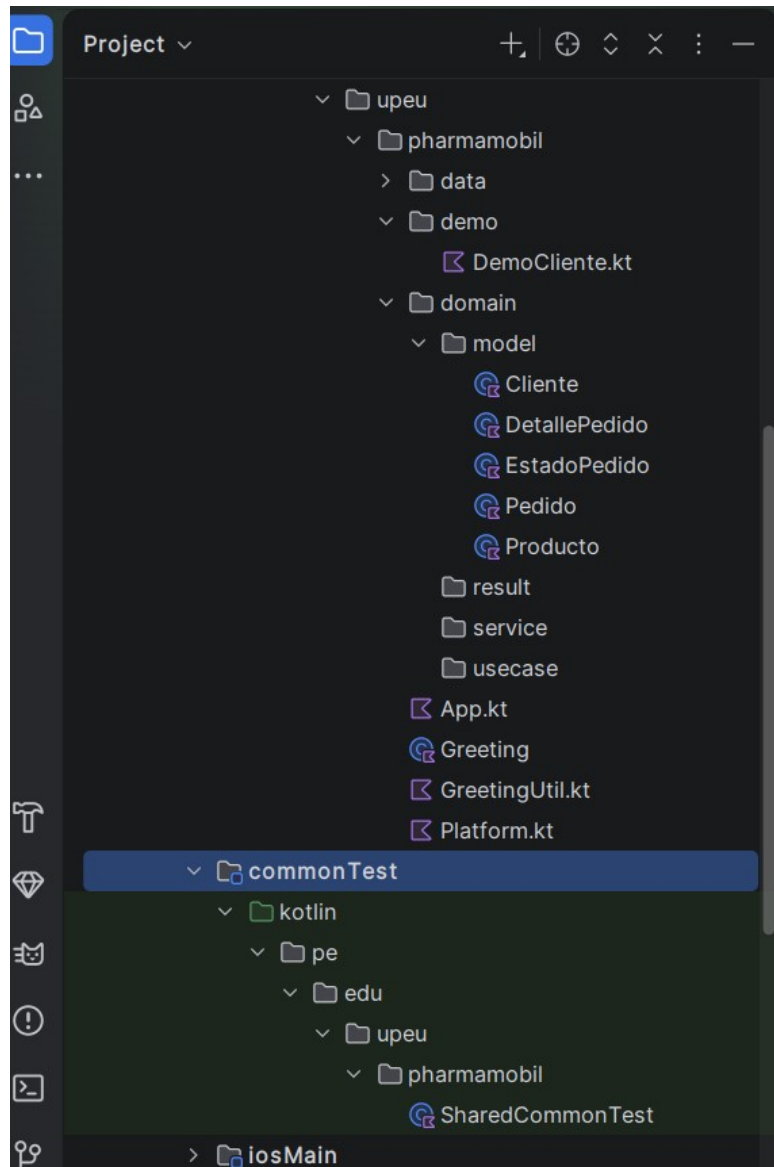


Figura 5. Ubicación del source set commonTest en el proyecto.

7. Matriz de verificación

Área	Escenario	Resultado esperado	Estado
Cliente	Teléfono registrado	987654321	Implementada
Cliente	Teléfono nulo	Sin teléfono	Implementada
Producto	Stock = 0	No disponible	Implementada
Producto	Stock suficiente	Disponible	Implementada
Detalle	12.50 × 3	37.50	Implementada
Detalle	Cantidad > stock	Excepción	Implementada
Pedido	Suma de subtotales	49.00	Implementada

8. Ejecución de las pruebas

En Android Studio, las pruebas deben ejecutarse como Android Host Test y no como prueba de dispositivo.

Windows PowerShell: `.\gradlew.bat :shared:testAndroidHostTest`

Linux/macOS: `./gradlew :shared:testAndroidHostTest`

9. Conclusiones

PharmaMobil cuenta con un modelo propio y consistente para una farmacia móvil. Las reglas de subtotal y stock están encapsuladas en el dominio y cubiertas por pruebas específicas, lo que facilita continuar con repositorios, servicios y pantallas sin duplicar lógica.

Entrega: el proyecto incluye un README en español con la estructura, las funcionalidades y los comandos necesarios para verificarlo desde el repositorio.